



Penetration Testing

1GOGH (CEX)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	25
Our Methodology	26
Disclaimers	28
About Hashlock	29

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The 1GOGH team partnered with Hashlock to conduct a security Penetration Test of their 1GOGH Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

1Gogh is an Australian-based next-generation cryptocurrency platform designed to enable users to buy, sell, store, and transfer digital assets seamlessly, particularly using Australian dollars (AUD). The platform aims to offer secure, fast, and user-friendly crypto transactions with minimal verification, supporting global transfers and business API integrations. It positions itself as a comprehensive ecosystem for trading and managing crypto assets while expanding features such as international transfers and automated payments

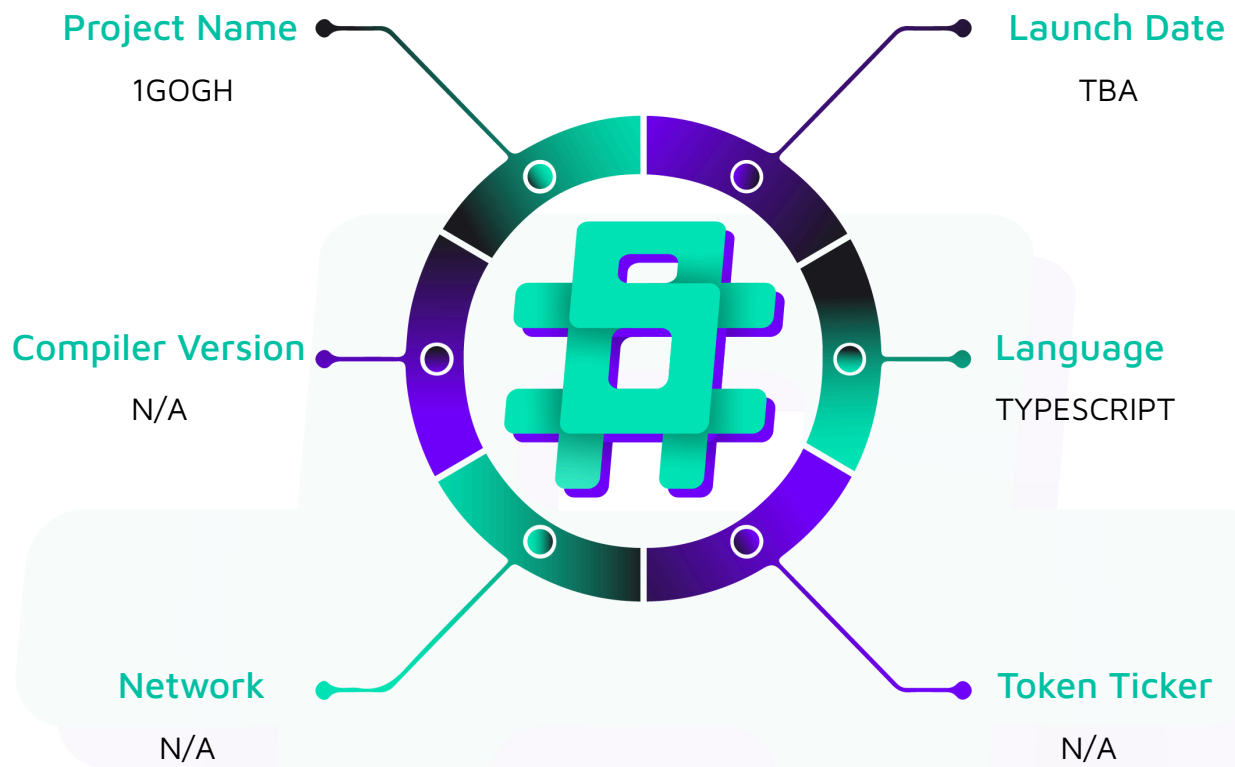
Project Name: 1GOGH

Project Type: CEX

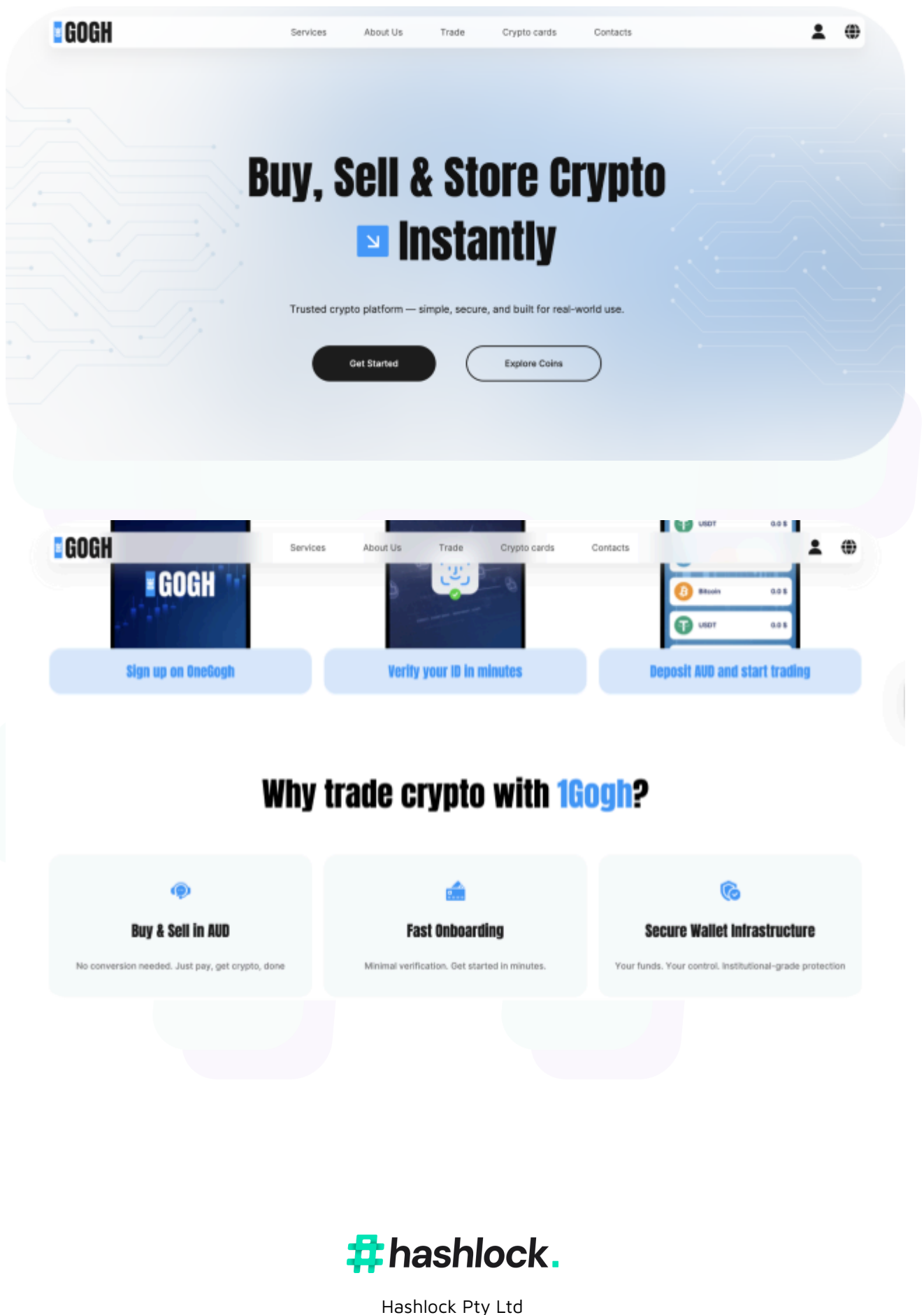
Website: <https://1gogh.io/>

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the 1GOGH project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	1GOGH Code Base
Platform	Web apps & APIs
Penetration Test Date	January, 2026
Repository/Website URL/IP Tested (1)	https://mobile.exchange.1gogh.io/
Repository/Website URL/IP Tested (2)	https://exchange.1gogh.io/
SSH server	Private address
AWS environment	AWS console access

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Medium-severity vulnerabilities

7 Low-severity vulnerabilities

2 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

Medium

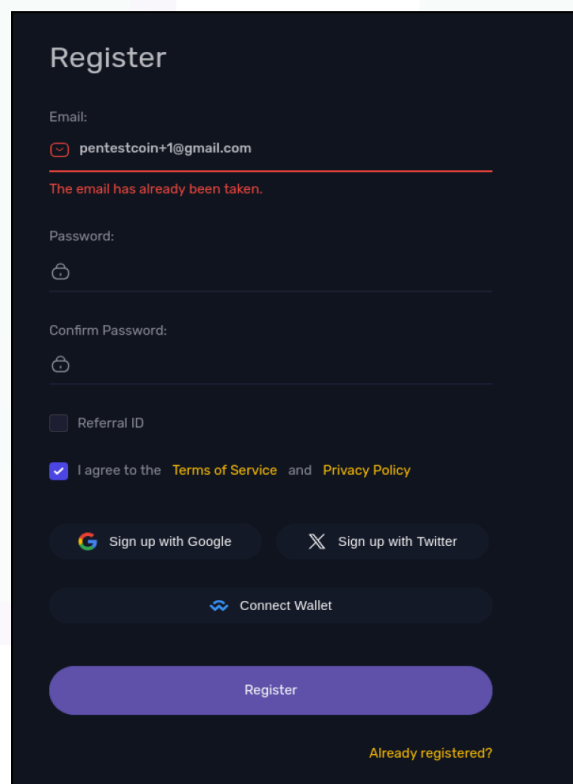
[M-01] 1gogh - Email enumeration

Description

Email enumeration enables attackers to identify valid email addresses for user accounts created within the application. This vulnerability occurs when application provide different types of messages during the process of registering or logging in e.g. *email already exist*.

Vulnerability Details

The application provides information about existing users during the registration process and change email functionality showing if the email address already exists. It clearly shows which email addresses are in use.

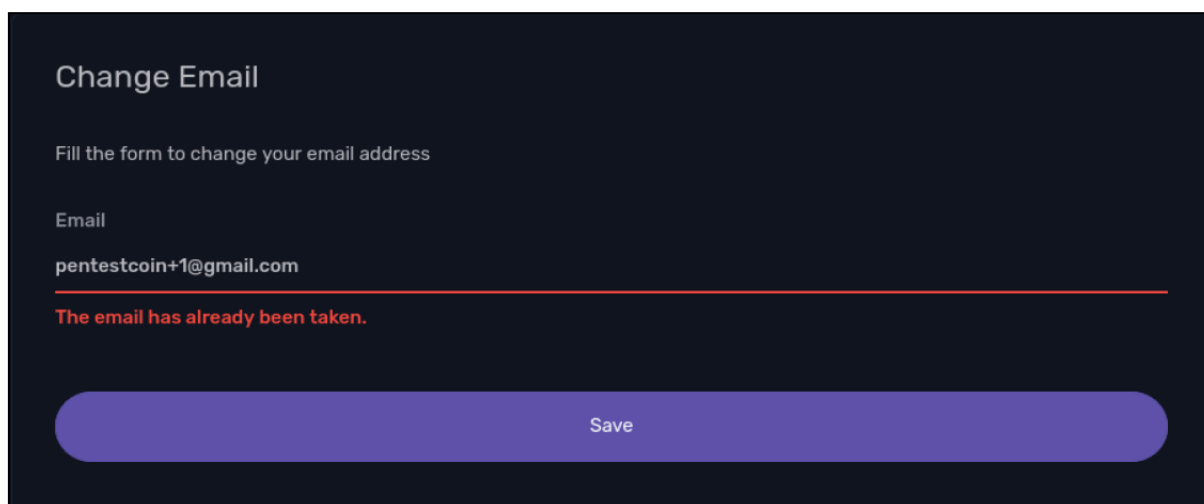


The screenshot shows a 'Register' form with the following fields and elements:

- Email:** A text input field containing 'pentestcoin+1@gmail.com'. Below the field, a red error message states: 'The email has already been taken.'
- Password:** A text input field with a password icon on the left.
- Confirm Password:** A text input field with a password icon on the left.
- Referral ID:** A checkbox that is currently unchecked.
- Agreement:** A checked checkbox followed by the text 'I agree to the [Terms of Service](#) and [Privacy Policy](#)'.
- Sign up with Google:** A button with the Google logo.
- Sign up with Twitter:** A button with the Twitter logo.
- Connect Wallet:** A button with a wallet icon.
- Register:** A large blue button at the bottom.
- Already registered?:** A link at the bottom right.

A similar situation happens on the change email form:





The screenshot shows a dark-themed web interface for changing an email address. At the top, the heading 'Change Email' is displayed. Below it, a subtitle reads 'Fill the form to change your email address'. A label 'Email' is positioned above the input field, which contains the text 'pentestcoin+1@gmail.com'. A red horizontal line underlines the input field, and a red error message 'The email has already been taken.' is displayed below it. At the bottom of the form is a large, rounded purple button labeled 'Save'.

Email enumeration significantly reduces the attack surface for credential-based attacks by providing attackers with confirmed, valid targets. With verified emails, attackers can focus brute force on known accounts rather than attempting random combinations. The vulnerability also enables social engineering attacks where attackers leverage confirmed usernames to craft convincing phishing campaigns.

Recommendation

It is recommended not to show the message displaying whether a user exists or not. Implement consistent error messages for all authentication scenarios, ensuring identical responses for both valid and invalid emails. If it is not possible to eliminate the existence of account feedback entirely, implement CAPTCHA to rule out automated email guessing.

Status

Resolved

[M-02] SSH server - Lack of privilege separation

Description

It was observed that the SSH server does not separate the regular user from the root user. The regular user was allowed to perform sudo without a password, which is equal to a privilege escalation attack.

The server however is reachable only after a whitelisting procedure, which greatly decreases potential attack surface. However, best security practice is to implement Principle of Least Privilege and separate daily usage from root that can access every resource on the server. In case of user compromise the attacker will be able to affect only user resources, but in the current situation, the whole server can be compromised as well.

Recommendation

It is recommended to implement a password policy on sudo at least. Ideally the user should be completely separated from root, however the latter is a long-term operation and would require auditing every sudo/suid enabled program to eliminate all privilege escalation vectors.

Status

Resolved

Low

[L-01] 1gogh - Missing security headers

Description

During the assessment of the exchange.1gogh.io, several security headers were found to be missing from HTTP responses:

- HTTP Strict Transport Security (HSTS) header enforces secure HTTPS connections, preventing protocol downgrade attacks and cookie hijacking. Recommended value: "max-age=31536000; includeSubDomains".
- Content Security Policy (CSP) protects against XSS attacks by specifying trusted content sources for scripts, styles, and other resources.
- X-Content-Type-Options prevents MIME-sniffing attacks by forcing browsers to honor the declared content-type. Required value: "nosniff".
- Referrer Policy controls information disclosure in navigations away from the site, enhancing user privacy.
- Permissions-Policy is a header that allows a site to control which features and APIs can be used in the browser.
- X-Frame-Options tells the browser whether you want to allow your site to be framed or not. By preventing a browser from framing your site you can defend against attacks like clickjacking. Recommended value "X-Frame-Options: SAMEORIGIN".

The absence of these headers increases exposure to XSS attacks, protocol downgrades, MIME-type confusion, and unintended information disclosure through referrer headers.

Recommendation

It is recommended to implement the missing security headers in the web server configuration or application code.

Status

Resolved

[L-02] 1gogh - Information disclosure of all API endpoints with parameters

Description

During the security assessment, it was found that there is an information disclosure of all API endpoints, including admin-associated functionalities with parameters and HTTP methods allowed. Despite the fact that there is a public API documentation under the path `/docs/api/`, there is an information disclosure in client-side javascript when users browse to non-existing endpoints and view the source code of the 404 page.

This can be viewed by entering any non-existent page, such as:

`https://exchange.1gogh.io/storage/uploads/.random.`

Then, in the page source, there will be a JavaScript variable named Ziggy with all API endpoints, HTTP methods, and parameters for each endpoint.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="csrf-token" content>
    <meta name="site-language" content="en">
    <title>1Gogh Exchange</title>
    <!-- Styles -->
    <link rel="stylesheet" href="/css/app.css">
    <link rel="stylesheet" href="/css/site.css">
    <!-- Scripts -->
  <script type="text/javascript">
    ...
    const Ziggy =
      {
        "url": "https://exchange.1gogh.io",
        "port": null,
        "defaults": {},
        "routes": {
          "login": {
            "uri": "login",
            "methods": ["GET", "HEAD"],
            "login.store": {
              "uri": "login",
              "methods": ["POST"],
              "logout": {
                "uri": "logout",
                "methods": ["POST"],
                "password.request": {
                  "uri": "forgot-password",
                  "methods": ["GET", "HEAD"],
                  "password.reset": {
                    "uri": "reset-password/{token}",
                    "methods": ["GET", "HEAD"],
                    "parameters": ["token"],
                    "password.email": {
                      "uri": "forgot-password",
                      "methods": ["POST"],
                      "password.update": {
                        "uri": "reset-password",
                        "methods": ["POST"],
                        "register": {
                          "uri": "register",
                          "methods": ["GET", "HEAD"],
                          "register.store": {
                            "uri": "register",
                            "methods": ["POST"],
                            "verification.notice": {
                              "uri": "email/verify",
                              "methods": ["GET", "HEAD"],
                              "verification.verify": {
                                "uri": "email/verify/{id}/{hash}",
                                "methods": ["GET", "HEAD"],
                                "parameters": ["id", "hash"],
                                "verification.send": {
                                  "uri": "email/verification-notification",
                                  "methods": ["POST"],
                                  "user-profile-information.update": {
                                    "uri": "user/profile-information",
                                    "methods": ["PUT"],
                                    "user-password.update": {
                                      "uri": "user/password",
                                      "methods": ["PUT"],
                                      "password.confirm": {
                                        "uri": "user/confirm-password",
                                        "methods":
```

```

> Ziggy
< ▾ {url: 'https://exchange.1gogh.io', port: null, defaults: {...}, routes:
  {...}} ⓘ
  ▸ defaults: {}
  port: null
  ▾ routes:
    ▸ admin.articles: {uri: 'exchange-control-panel/articles', methods: A
    ▸ admin.articles.create: {uri: 'exchange-control-panel/articles/creat
    ▸ admin.articles.destroy: {uri: 'exchange-control-panel/articles/{art
    ▸ admin.articles.edit: {uri: 'exchange-control-panel/articles/{articl
    ▸ admin.articles.restore: {uri: 'exchange-control-panel/articles/{art
    ▸ admin.articles.store: {uri: 'exchange-control-panel/articles', meth
    ▸ admin.articles.update: {uri: 'exchange-control-panel/articles/{arti
    ▸ admin.bank_accounts: {uri: 'exchange-control-panel/bank-accounts',
    ▸ admin.bank_accounts.create: {uri: 'exchange-control-panel/bank-acco
    ▸ admin.bank_accounts.destroy: {uri: 'exchange-control-panel/bank-acc
    ▸ admin.bank_accounts.edit: {uri: 'exchange-control-panel/bank-accoun
    ▸ admin.bank_accounts.store: {uri: 'exchange-control-panel/bank-accou
    ▸ admin.bank_accounts.update: {uri: 'exchange-control-panel/bank-acco
    ▸ admin.cold_storage: {uri: 'exchange-control-panel/cold-storage', me
    ▸ admin.cold_storage.create: {uri: 'exchange-control-panel/cold-stora
    ▸ admin.cold_storage.destroy: {uri: 'exchange-control-panel/cold-stor
    ▸ admin.cold_storage.edit: {uri: 'exchange-control-panel/cold-storage
    ▸ admin.cold_storage.store: {uri: 'exchange-control-panel/cold-storag
    ▸ admin.cold_storage.update: {uri: 'exchange-control-panel/cold-stora
    ▸ admin.currencies: {uri: 'exchange-control-panel/currencies', method
    ▸ admin.currencies.coinpayments.coins: {uri: 'exchange-control-panel/
    ▸ admin.currencies.coinpayments.sync: {uri: 'exchange-control-panel/c
    ▸ admin.currencies.create: {uri: 'exchange-control-panel/currencies/c
    ▸ admin.currencies.destroy: {uri: 'exchange-control-panel/currencies/
    ▸ admin.currencies.edit: {uri: 'exchange-control-panel/currencies/{cu
    ▸ admin.currencies.restore: {uri: 'exchange-control-panel/currencies/
    ▸ admin.currencies.store: {uri: 'exchange-control-panel/currencies',
    ▸ admin.currencies.update: {uri: 'exchange-control-panel/currencies/{
    ▸ admin.dashboard: {uri: 'exchange-control-panel', methods: Array(2)}
    ▸ admin.kyc.documents: {uri: 'exchange-control-panel/kyc-documents',
    ▸ admin.kyc.moderate: {uri: 'exchange-control-panel/kyc-documents/{do
    ▸ admin.language.translations: {uri: 'exchange-control-panel/language
    ▸ admin.language.translations.destroy: {uri: 'exchange-control-panel/
    ▸ admin.language.translations.store: {uri: 'exchange-control-panel/la
    ▸ admin.language.translations.sync: {uri: 'exchange-control-panel/lan
    ▸ admin.language.translations.update: {uri: 'exchange-control-panel/l
    ▸ admin.languages: {uri: 'exchange-control-panel/languages', methods:
    ▸ admin.languages.create: {uri: 'exchange-control-panel/languages/cre

```

Recommendation

Remove or minimize the Ziggy routing configuration from client-side code by implementing filtering that only exposes routes required by the frontend application. Configure the Ziggy to exclude administrative, internal, and privileged endpoints from the JavaScript object.

Status

Unresolved

[L-03] 1gogh - 2FA is not enforced during password/email change while the user is authenticated

Description

There is a misconfiguration in the 2FA enforcement mechanism because the second authentication factor is not enforced during a session when significant changes are made to the account e.g. password/email change.

During the penetration test, it was found that the application does not enforce the provision of a second factor of authentication when a user updates their password while logged in.

Recommendation

While two-factor authentication is enabled, it is recommended that all profile updates are made after providing the second factor of authentication, especially the ones associated with authentication.

Status

Resolved

[L-04] 1gogh - Overly Permissive Cross-Origin Resource Sharing policy

Description

The application reflects any origin supplied in the request's `Origin` header back in the `Access-Control-Allow-Origin` response header, while also including `Access-Control-Allow-Credentials: true`. Such a behaviour allows any website to make authenticated cross-origin requests to the application. Although direct exploitation through CSRF-via-CORS was prevented due to only non-sensitive cookies being transmitted.

While immediate exploitation is mitigated by proper cookie flags, this misconfiguration poses risks. If session cookie configurations change in the future (e.g., `SameSite` attribute removed or set to `None`), the application becomes immediately vulnerable to cross-origin attacks. Any sensitive data returned in API responses can be read by malicious websites. Additionally, non-sensitive cookies being transmitted could still leak information or be chained with other vulnerabilities.

Recommendation

Implement a strict CORS policy with an explicit allowlist of trusted origins. If credentials are required, ensure the `Access-Control-Allow-Origin` header specifies exact origins rather than using wildcards or reflection.

Status

Resolved

[L-05] 1gogh - Predictable object identifiers

Description

The application uses sequential, easily predictable user identifiers in the account verification process. Newly created accounts receive simple numeric IDs (e.g., 61), which are then exposed both in the verification link and in the API response from `/user/profile` endpoint.

During account creation, the application generates IDs in the form of sequential integers. This identifier is visible in the verification URL:

```
https://exchange.1gogh.io/email/verify/61/ecf15e8871e49b890c7b065b876e6498ab90c1c6?expires=1768341246&signature=7ec01c03f973924a316a9bdc162a652c1ab27eee46bff509177c27d81dfb3c0d
```

and in the HTTP response body from `/user/profile` as the `id` field.

```
HTTP/2 200 OK

Content-Type: application/json

Date: Fri, 16 Jan 2026 11:07:22 GMT

Vary: X-Inertia,Origin

Cache-Control: no-cache, private

Set-Cookie: REDACTED

Set-Cookie: REDACTED

Set-Cookie: REDACTED

Set-Cookie: REDACTED

Server: Apache/2.4.65 (Ubuntu)

X-Inertia: true

X-Cache: Miss from cloudfront

Via: 1.1 4c0149793a766b424f3ddc1372e41924.cloudfront.net (CloudFront)

X-Amz-Cf-Pop: WAW51-P4
```

```
Alt-Svc: h3=":443"; ma=86400

X-Amz-Cf-Id: T2KDofWMsx-LSDEX1KI-cj_rvuEatpZNUDcz36UqhD1jeRNKXChrgQ==

{"component":"Profile\Show","props":{"errors":{},"jetstream":null,"auth":{"user":{"id
":61,"name

[...]
```

After creating an account one by one, we got two verification emails showing the sequential nature of these identifiers, which is easily predictable through simple incrementation.

Predictable identifiers facilitate user enumeration attacks and can be leveraged as a supporting vector in other attacks, such as IDOR or targeted phishing campaigns.

While predictable IDs alone are not a vulnerability without additional flaws, they increase the application's attack surface, as it is possible to easily map objects' IDs by simple incrementation.

Recommendation

It is recommended to replace sequential numeric identifiers with UUID/GUID values (e.g., 550e8400-e29b-41d4-a716-446655440000) in all public API interfaces and URLs.

Status

Acknowledged

[L-06] AWS - MFA not enforced for all human IAM users**Description**

Multi-factor authentication (MFA) is not enforced for all human IAM users, allowing authentication using single-factor credentials. It was possible to use the audit account without enforcement of MFA. Without MFA, compromise of a single IAM user credential enables direct access to AWS management interfaces and APIs within the scope of the user's permissions.

Recommendation

Enforce MFA for all human IAM users by attaching MFA-enforcing IAM policies.

Status

Resolved

[L-07] AWS - Internet-exposed instances lack runtime monitoring telemetry**Description**

All observed EC2 instances with public IPv4 addresses are running with monitoring disabled. This reduces available telemetry for CPU, network, and disk behavior, which are commonly used to detect abuse such as tunneling, brute-force activity, or lateral movement after initial access.

Without monitoring, malicious activity occurring after instance compromise may persist longer without detection, increasing attacker dwell time and operational impact.

Recommendation

Enable detailed EC2 monitoring and integrate metrics with alerting to detect anomalous runtime behavior on internet-facing instances.

Status

Resolved

QA

[Q-01] 1gogh - Account created with only SSO can't perform some actions

Description

The application requires a password to be provided for actions such as deleting an account, enabling two-factor authentication (2FA) or logging out of other browser sessions. There is a conflict if the account was only created using social login, such as a Google account.

The application still requires the password, but the user doesn't have one, so they can't perform the aforementioned actions.

Recommendation

It is recommended that this logic be revised and a different mechanism be implemented, which will enable users to perform crucial actions after logging in. Alternatively, the user could be prompted to set up a password after logging in.

Status

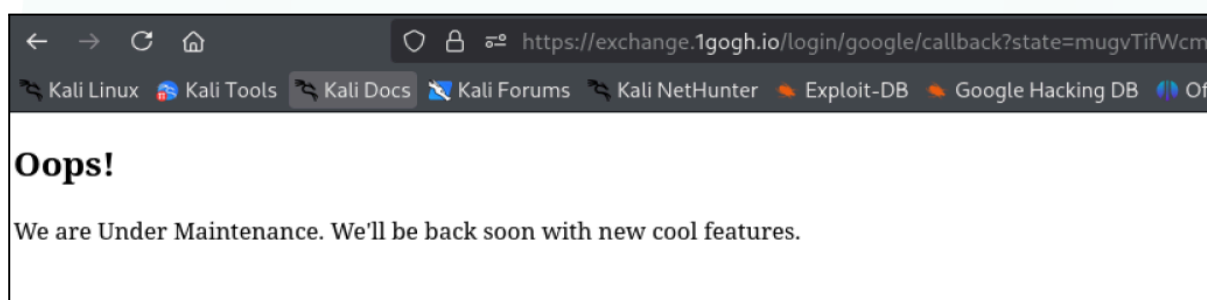
Resolved

[Q-02] 1gogh - Deleting an account linked to a social login will result in an inability to use that same social account in the future

Description

There is an issue with the deletion mechanism: the application does not properly validate deleted accounts. If an account linked to social login has been deleted, it is not possible to create a new account using that same social account even after a few days. The account was deleted on 13 January, but it was not possible to create an account with the same Google account on 16 January.

When trying to create the account using social login with the same account we got a 500 Internal Server Error message and information like the following on the client side.



Recommendation

Improve the deletion mechanism so that, after a successful deletion, it is possible to create an account using the same social account.

Status

Unresolved

Conclusion

After Hashlock's analysis, the 1GOGH project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.